How To Start And Adapt A New Project

A plain-language guide to using AEGIS in your own project. No prior knowledge of the template internals is needed.

If you only remember one thing: copy the template into your repo, then edit one file — `.agentic/workflow.toml` — to match your project. Everything else is optional polish.

1. What is this, in one paragraph?

This template gives an AI coding agent (Codex, Claude Code, Pi, opencode, or Qwen Code — adapters ship for all five) a disciplined way to work on your project. Instead of letting the agent change code freely, the work is broken into seven gates. At each gate the agent stops, shows you what it did, and waits for your approval. Intent Alignment is the independent gate after intake where the agent records what it understood, how that relates to the current software, open questions with default responses, risk, and the planned path. Every behavior change must still be tied to a written requirement and a test, so later work cannot quietly undo earlier work. You stay in control the whole time.

```
flowchart LR
  A[Your idea or bug] --> B[Intake]
  B --> C[Intent Alignment]
  C --> D[Requirement]
  D --> E[Check / test]
  E --> F[Implementation]
  F --> G[Validation]
  G --> I[Delivery]
  I --> H[You approve PR / commit]

  classDef gate fill:#1f6feb22,stroke:#1f6feb,color:#111;
  class B,C,D,E,F,G,I gate;
```

You drive it with normal chat prompts. The template adds a small command-line tool, some guardrail "hooks", and instructions the agent reads automatically.

2. What you need before you start

You need	Why	How to check
Python 3.11+ runtime	Runs the workflow tool and guardrail hooks; packaged WAEGIS registers one automatically	Run `.agentic/tools/run.ps1 --help` or `.agentic/tools/run.sh --help`
Node.js	Every guardrail hook (write-zone guard, secret scan, ...) is dispatched through it via `.agentic/tools/run_hook.mjs`	`node --version`
Git	The tool uses git to scan changes, find secrets, and gate delivery	`git --version`
An AI agent	Codex, Claude Code, Pi, opencode, or Qwen Code (the template ships adapters for all five; Codex/Claude Code/Pi are the most exercised)	—

> Without git, the delivery check fails on purpose and you cannot finish a run.
> Always work inside a git repository.

> Python and Node are requirements of the AEGIS/WAEGIS tooling itself, not of your project. They apply no matter what language your own project is written in — C, C++, embedded firmware, Java, Go, whatever. Your project's own compiler/build system/test runner is entirely separate and unaffected; AEGIS only asks it to run gate commands you configure yourself in `.agentic/workflow.toml` (`[gates] test` / `[gates] lint`; WAEGIS answers use corresponding setup fields internally). A machine that can already build your project may still need Python and Node installed

just to run AEGIS's own workflow tool and hooks. `waegis new`/`waegis adopt` check for both automatically (Python before proceeding; Node is checked and reported, but never blocks setup — see `prereq.node` in the wizard's plan preview).

3. The big picture: what gets added to your repo

When you copy the template in, your project gains these folders/files. You rarely touch most of them.

```
flowchart TB
  subgraph YOURS["Your project (unchanged)"]
    SRC["your source code"]
    TESTS["your tests"]
  end

  subgraph ENGINE[".agentic  -  the engine you mostly leave alone"]
    CLI[".agentic/cli  - the workflow tool"]
    TOOLS[".agentic/tools - run / install scripts"]
    HOOKS[".agentic/hooks - guardrails"]
    SKILLS[".agentic/skills - shared instructions"]
    REQS[".agentic/requirements - your durable requirements"]
    CONF[[".agentic/workflow.toml  - YOU EDIT THIS"]]
  end

  subgraph ADAPTERS["Agent adapters"]
    CODEX[".codex  - for Codex"]
    CLAUDE[".claude + CLAUDE.md - for Claude Code"]
    PI[".pi  - for Pi"]
    OPENCODE[".opencode + opencode.json - for opencode"]
    QWEN[".qwen + QWEN.md - for Qwen Code"]
    AGENTS["AGENTS.md - the shared rulebook"]
  end

  CONF -. "tells the engine your roots, tests, gates" .-> ENGINE

  classDef edit fill:#f8514922,stroke:#f85149,color:#111;
  class CONF edit;
```

The only file you must edit is highlighted: `.agentic/workflow.toml`.

4. Step-by-step

Step 1 — Copy the template into your project

From this template repository, run the helper (replace the path with your new project folder):

```
python 02_Tools\create_project_from_template.py D:\Path\To\NewProject
```

Or simply copy everything inside `05_Software/aegis/template/` into your project's root folder by hand. After this, your project root contains `.agentic/`, `.codex/`, `.claude/`, `.pi/`, `.opencode/`, `.qwen/`, `AGENTS.md`, `CLAUDE.md`, `QWEN.md`, `opencode.json`, and a few docs.

Step 2 — Make sure it is a git repository

```
cd D:\Path\To\NewProject
git init        # skip if your project is already a git repo
```

Step 3 — Check the tool runs

```
.\.agentic\tools\run.ps1 --help
```

On Linux/macOS use `bash .agentic/tools/run.sh --help` instead. If you see the list of commands, you are ready. (Workflow commands can later be run from any subfolder of the repo — the tool finds the project root by itself.) (Optional: run `.\.agentic\tools\install.ps1` to install a faster `aegis` command — not required.)

Step 4 — Adapt `.agentic/workflow.toml` (the important step)

This file tells the engine where your code, tests, and docs live, and what commands to run. Open it and adjust it to your project.

The defaults assume source in `src/`, tests in `tests/`, and Python. Here is a worked example for a project called *invoicer* whose code lives in `invoicer/` and whose tests live in `tests/`:

```
[source]
roots = ["invoicer"]          # was ["src", "app", "lib"]

[tests]
roots = ["tests"]

[traceability]
language = "python"
source_extensions = [".py"]
coverable_roots = ["invoicer"] # keep this the same as [source] roots

[gates]
# These start EMPTY. Put your real commands here, or "gates run" does nothing.
test = ["python -m pytest -q"]
lint = ["python -m ruff check ."]

[write_zones]
# IMPORTANT: keep these aligned with the roots above (see the gotcha below).
implementation = ["invoicer/**", ".agentic/knowledge/components/**"]
check = ["tests/**"]
```

Things you will commonly change:

Setting	What it means	Example
`[source] roots`	Where your code lives	`["invoicer"]`
`[tests] roots`	Where your tests live	`["tests"]`
`[gates] test / lint`	Real commands to validate the project (default empty)	`["python -m pytest -q"]`
`[traceability] language`	How linked tests are matched: `python`, `java`, `c`, `cpp`, or `other`	`"python"`
`coverable_roots`	Which code files must be tied to a requirement	`["invoicer"]`
`[secrets] patterns`	Token shapes that must never be committed	keep the defaults
`[write_zones]`	Which files the agent may edit at each gate	see gotcha

The one gotcha: keep `write_zones` aligned with your roots

A guardrail stops the agent from editing the "wrong" files at each gate. During the implementation gate it may only touch the paths listed in `write_zones.implementation`. If your code is in `invoicer/` but that list still says `src/`, the agent will be blocked from editing its own code and you will see confusing "edit outside write zone" messages.

> Rule of thumb: whenever you change `[source] roots`, change `write_zones.implementation` to match. Whenever you change `[tests] roots`, change `write_zones.check` to match.

Step 5 — (Optional) glance at the agent adapters

You usually do not need to edit these:

- Claude Code — `.claude/settings.json` already wires the guardrails and

blocks pushing/PRs by default. Leave it unless you want different rules.

- Codex — `.codex/config.toml` runs offline by default

(`network.enabled = false`). Leave it offline unless your project genuinely needs network access and you approve it.

- Pi — `.pi/` ships a bridge extension that wires the same guardrails. Run

`pi install ./.pi` and trust the project on first start. The skills surface as `/skill:<name>`. See `.pi/README.md`. Python must be on your PATH.

• opencode — `.opencode/` plus the root `opencode.json` wire the same

guardrails through an opencode plugin. See `.opencode/README.md`.

• Qwen Code — `.qwen/` wires the same guardrails; `settings.json` loads

`AGENTS.md` first and treats `QWEN.md` only as the adapter note. See `.qwen/README.md`.

• `AGENTS.md`, `CLAUDE.md`, `QWEN.md`, and the skills are the shared rulebook —

leave them as-is.

Step 6 — Let the agent sanity-check your setup

Open your new project in your agent tool (Codex, Claude Code, Pi, opencode, or Qwen Code) and paste this prompt. It does not change anything — it just verifies your `workflow.toml` matches the real project:

```
Use the agentic development workflow installed in this repository.
Read AGENTS.md and .agentic/workflow.toml.
Inspect the configured source, test, requirement, and documentation roots.
Tell me whether the configuration matches this project and list any paths or
gate commands that need adjustment. Do not edit files yet.
```

Fix anything it flags, and you are set up.

5. Using it day to day

Once configured, you just talk to the agent. A typical first task:

```
Use the agentic development workflow.
Implement feature: add a 10% discount for orders over 100 EUR.
Stop at each gate and wait for my approval.
```

The agent then walks the seven gates, stopping after each one:

```
flowchart LR
  I[Intake<br/>start the run] --> A[Alignment<br/>record understanding]
  A --> R[Requirement<br/>write it down]
  R --> C[Check<br/>add a failing test]
  C --> M[Implementation<br/>make the test pass]
  M --> V[Validation<br/>run all checks]
  V --> D[Delivery<br/>preflight + your OK]
```

What each gate means in plain terms:

• Intake - the agent starts the run and identifies the likely request type.

• Intent Alignment - it records understanding, scope, assumptions, open

questions with recommended default responses, risk, decision, reason, and planned paths. This gate can be skipped with `[alignment].enabled = false`.

• Requirement - it writes the requirement down (in

`.agentic/requirements/`). You approve the wording.

• Check - it adds a test that *fails first* (proving the test is real).

• Implementation - it writes code until that test passes, and only touches

allowed files.

• Validation - it runs your gate commands, audits, and reviews the change.

• Delivery - it runs a final preflight and asks before committing/pushing.

You can stop, revise, or abort at any gate. Nothing is pushed or committed without your explicit go-ahead.

`PROMPT_CHEATSHEET.md` (copied into your project) has ready-to-use prompts for every gate.

6. Quick checklist

```
[ ] Copy template/ into the new project root
```

```
[ ] git init (if needed)
[ ] A registered/system Python 3.11+ runtime and git available
[ ] Node.js available (every guardrail hook runs through it - warned, not
      blocked, if missing)
[ ] run.ps1 --help works
[ ] Edit .agentic/workflow.toml:
      [ ] [source] roots        -> your code folders
      [ ] [tests] roots         -> your test folders
      [ ] coverable_roots       -> same as [source] roots
      [ ] [gates] test / lint   -> your real commands
      [ ] [traceability] language + source_extensions -> your language
      [ ] write_zones.implementation -> matches [source] roots  (+ .agentic/knowledge/components/**)
      [ ] write_zones.check          -> matches [tests] roots
[ ] (Optional) review .codex/config.toml, .claude/settings.json, .pi/README.md,
      .opencode/README.md, and .qwen/README.md for the adapter you use
[ ] (Pi only) pi install ./.pi and trust the project
[ ] Run the "sanity-check" prompt with your agent
[ ] Start your first feature
```

7. Adapting to a different layout or language

The example above is Python. For other setups:

• Different folders — only the names change. A Java project with code in

`src/main/java` and tests in `src/test/java` sets those as `[source] roots` and `[tests] roots`, and mirrors them in `write_zones`.

• Different language — set `[traceability] language` to `java`, `c`, or

`cpp` for smarter test matching. Anything else falls back to a simple name-match, which still works. Set `source_extensions` (e.g. `[".ts"]`) and `test_file_patterns` to fit.

• Docs elsewhere — if you keep component docs somewhere other than

`.agentic/knowledge/components/`, update `[documentation] roots`, `component_template`, and the `.agentic/knowledge/components/` entry in `write_zones.implementation`.

8. Troubleshooting

Symptom	Likely cause	Fix
"Edit outside write zone" when the agent tries to edit your code	`write_zones.implementation` does not include your source folder	Align it with `[source] roots`
`gates run` does nothing	`[gates] test` / `lint` are still empty	Add your real commands
Delivery preflight always fails	Not inside a git repo, or git not installed	`git init`; install git
Hooks do nothing / not enforced	No explicit, WAEGIS-registered, or system Python runtime is available	Run packaged WAEGIS once, set `AEGIS_PYTHON`, or install Python 3.11+
Hooks error, or your agent tool reports a failed/missing hook command	Node.js is not installed (every hook is dispatched through `node .agentic/tools/run_hook.mjs`)	Install Node.js; `waegis`'s `prereq.node` step reports this too, but never blocks setup
Coverage audit complains about a file	A new code file has no linked requirement	Add a requirement row, or waive legacy files in `.agentic/requirements/coverage_waivers.txt`

9. Where to read more

• `AGENTS.md` — the authoritative rulebook the agent follows.

• `CLI_API_REFERENCE.md` — every workflow command, with examples (includes the

runtime requirements).

• `PROMPT_CHEATSHEET.md` — copy-paste prompts for each gate.

• `WORKFLOW_CONFIGURATION.md` — configuration details for roots, gates,

traceability, matching semantics, and write zones.

- `COMPONENTS_CONFIGURATION.md` — component maps, adoption baseline, and component documentation behavior.